APUNTES COMPLETOS Y BLINDADOS (TEST 1) — WEB TECHNOLOGIES I

Cobertura 100% Diapositivas (Temas 1 a 6): HTML5 + Formularios + CSS3 + Box Model + Positioning + Selectors + Flex/Grid + Responsive

10 puntos de 13 = **10/10 Nota Máxima**

1. Task 1: Todos los Errores de Sintaxis HTML5 Posibles — Task 1 • 1 pt

En la **Task 1** te pedirán buscar y corregir **1 error de sintaxis** en un bloque de ~12 líneas. Sigue esta lista de comprobación (de más probable a menos probable):

1. **CASO 1 (EXAMEN EJEMPLO) Elemento de bloque dentro de `<p>`**:
   Un párrafo `<p>` **JAMÁS puede contener elementos de bloque** (ni siquiera otro `<p>`, ni `<h1>..<h6>`, `<div>`, `<ul>`, `<ol>`, `<dl>`, `<table>`, `<form>`, `<hr>`, `<blockquote>`).
   ❌ `<p class="heading"><h1>First</h1></p>`
   ✅ **Solución:** Quitar `<p>` (`<h1 class="heading">First</h1>`) o cambiar `<p>` por `<div>`.
2. **CASO 2 Etiquetas mal anidadas / solapadas (Overlapping tags)**:
   La última etiqueta que se abre debe ser la primera en cerrarse (LIFO).
   ❌ `<p><strong>Texto <em>importante</strong></em></p>`
   ✅ **Solución:** `<p><strong>Texto <em>importante</em></strong></p>`.
3. **CASO 3 Elementos Vacíos (Void Elements) con etiqueta de cierre o con contenido**:
   Lista completa de Void Elements en HTML5: `<area>`, `<base>`, `<br>`, `<col>`, `<embed>`, `<hr>`, `<img>`, `<input>`, `<link>`, `<meta>`, `<source>`, `<track>`, `<wbr>`.
   ❌ `<img src="a.jpg" alt="A"></img>` | ❌ `<input type="text">Nombre</input>` | ❌ `<br></br>`.
4. **CASO 4 Hijos directos inválidos en Listas (`ul, ol, dl`) o Tablas (`table, tr`)**:
   • Dentro de `<ul>` y `<ol>` **SOLO** puede haber `<li>` como hijos directos (❌ meter un `<p>` o `<h2>` suelto entre dos `<li>`; debe ir *dentro* del `<li>`).
   • Dentro de `<dl>` solo puede haber `<dt>` y `<dd>` (o un `<div>` que agrupe pares `dt/dd`).
   • Dentro de `<tr>` **SOLO** puede haber `<td>` o `<th>`.
5. **CASO 5 Elementos interactivos anidados**:
   ❌ Un `<a>` dentro de otro `<a>`, o un `<button>` / `<input>` dentro de un `<a>`, o un `<form>` dentro de otro `<form>`.
   Nota: En HTML5 `<a>` **SÍ puede envolver bloques** (ej. `<a href="..."><div>...</div></a>` es válido en HTML5 siempre que no contenga otros elementos interactivos).
6. **CASO 6 Errores en Atributos o Estructura del Documento**:
   • Dos elementos con el mismo `id="menu"` (`id` debe ser único en todo el documento; `class` sí se puede repetir).
   • Comillas sin cerrar en un atributo (ej. `class="heading>`) o atributos separados sin espacio (`id="a"class="b"`).
   • `<img>` sin el atributo obligatorio `alt="..."` o sin `src="..."`.
   • `<title>` fuera de `<head>`, o etiquetas visuales (`<h1>`, `<p>`, `<div>`) colocadas dentro de `<head>` en vez de `<body>`.
   • Más de un elemento `<main>` visible en la página.

Tabla Completa: Elementos Block-level vs Inline

Block-level (Empiezan en nueva línea, ocupan 100% ancho disponible, aceptan width/height/margin/padding)	Inline (Fluyen en la misma línea de texto, ancho = su contenido, IGNORAN width/height y margin top/bottom)
• **Semánticos de estructura:** `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>` • **Texto y contenedores:** `<div>`, `<p>`, `<h1>..<h6>`, `<blockquote>`, `<pre>`, `<figure>`, `<figcaption>`, `<hr>`, `<address>` • **Listas, tablas y forms:** `<ul>`, `<ol>`, `<li>`, `<dl>`, `<dt>`, `<dd>`, `<table>`, `<form>`, `<fieldset>`	• **Texto semántico:** `<span>`, `<a>`, `<strong>`, `<b>`, `<em>`, `<i>`, `<u>`, `<s>`, `<small>`, `<mark>`, `<sub>`, `<sup>`, `<code>`, `<kbd>`, `<samp>`, `<var>`, `<q>`, `<cite>`, `<abbr>`, `<time>`, ` `, `<wbr>` • **Reemplazados / Controles (inline-block por comportamiento):** `<img>`, `<input>`, `<label>`, `<select>`, `<textarea>`, `<button>`, `<iframe>`, `<audio>`, `<video>`

2. Task 2: Semántica, Listas, Entidades y URLs — Task 2 • 2 pts

A. Elementos Semánticos de Sección HTML5 (Diapositiva 03)

Etiqueta	Significado Exacto (para explicar en el examen)
`<header>`	Cabecera de la página o de una sección/artículo (suele contener logo, título, navegación).
`<nav>`	Bloque principal de enlaces de navegación.
`<main>`	Contenido principal y único de la página (no se repite entre páginas). Solo 1 por documento.
`<article>`	Contenido independiente y autocontenido (post de blog, noticia, producto) que tiene sentido por sí solo.
`<section>`	Agrupación temática de contenido, normalmente con un encabezado (`<h1>..<h6>`).
`<aside>`	Contenido secundario o barra lateral (publicidad, enlaces relacionados, glosario).
`<footer>`	Pie de página o de sección (autoría, copyright, contacto, enlaces legales).
`<figure>` y `<figcaption>`	Contenedor para ilustraciones, fotos o código con su pie de foto (`<figcaption>`).

B. Los 3 Tipos de Listas en HTML

- `<ul>` **(Unordered List):** Lista con viñetas/puntos (`disc` por defecto). Hijos: `<li>`.
- `<ol>` **(Ordered List):** Lista numerada. Atributos importantes:
  - `type="1 | A | a | I | i"` (números, letras mayús/minús, números romanos).
  - `start="5"` (empieza a contar desde el 5).
  - `reversed` (cuenta hacia atrás: 5, 4, 3...).
  - En un `<li value="10">` cambia el número de ese ítem y los siguientes.
- `<dl>` **(Description List):** Lista de pares término-descripción.
  - `<dt>` (*Description Term*): El nombre o concepto (sin sangría).
  - `<dd>` (*Description Details*): La definición o valor (se muestra en línea nueva con **sangría/margen izquierdo de ~40px**). Puede haber varios `<dt>` para un `<dd>` o varios `<dd>` para un `<dt>`.

C. Entidades HTML y Resolución de URLs

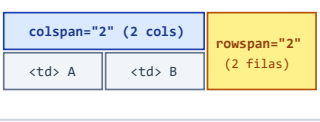
Entidad	Símbolo	Por qué es obligatorio usarla
`<` / `<`	<	Evita que el navegador interprete el signo menor que como apertura de etiqueta HTML.
`>` / `>`	>	Signo mayor que (cierre de etiqueta).
`&` / `&`	&	Evita que `&` se interprete como el inicio de otra entidad HTML.
`"` / `'`	" / '	Para incluir comillas dentro de valores de atributos delimitados por esas mismas comillas.
` `	`[espacio]`	*Non-breaking space*: impide salto de línea entre dos palabras y permite poner varios espacios seguidos (HTML colapsa los espacios normales a 1 solo).

```
Página actual: http://www.avatar.com/movie/html/index.html
• banner.jpg o ./banner.jpg → http://www.avatar.com/movie/html/banner.jpg (misma carpeta)
• ../img/a.jpg → http://www.avatar.com/movie/img/a.jpg (sube 1 nivel: sale de /html/)
• ../../images/banner.jpg → http://www.avatar.com/images/banner.jpg (sube 2 niveles)
• /images/banner.jpg (empieza por /) → http://www.avatar.com/images/banner.jpg (raíz del domir
```

3. Tablas HTML (`colspan` / `rowspan`), Enlaces y Mapas de Imagen — Por si sale en Task 1 o Task 2

A. Estructura de Tablas y Fusión de Celdas

- **Etiquetas de tabla:** `<table>` (contenedor), `<caption>` (título de la tabla, debe ser el 1er hijo de `table`), `<thead>` (cabecera), `<tbody>` (cuerpo), `<tfoot>` (pie), `<tr>` (fila / *table row*), `<th>` (celda de encabezado: en **negrita y centrada** por defecto), `<td>` (celda de datos: alineada a la izquierda por defecto).
- `colspan="N"` **(Fusión Horizontal):** La celda ocupa **N columnas hacia la derecha** en esa misma fila. (En esa fila habrá `N-1` etiquetas `<td>` menos).
- `rowspan="N"` **(Fusión Vertical):** La celda ocupa **N filas hacia abajo**. (En las siguientes `N-1` filas de abajo habrá 1 etiqueta `<td>` menos).




```
Fila 1: <tr><td colspan="2">..</td><td rowspan="2"
Fila 2: <tr><td>A</td><td>B</td></tr>
¡En Fila 2 solo se escriben 2 <td> porque la 3ª viene del rows
```

B. Enlaces (`<a>`), Mapas de Imagen y Multimedia

- **Tipos de `href` en `<a>`:**
  - `href="#id"` → Salta (scroll) al elemento con `id="id"` en la **misma página**.
  - `href="otra.html#id"` → Carga `otra.html` y hace scroll hasta `id="id"`.
  - `href="mailto:user@lu.lv?subject=Hi"` → Abre el programa de email.
  - `href="tel:+37120000000"` → Inicia llamada telefónica (en móviles).
  - Atributo `download="nombre.pdf"` → Fuerza la descarga del archivo en vez de abrirlo en el navegador.
  - Atributo `target="_blank"` (nueva pestaña) vs `target="_self"` (misma pestaña, por defecto).
- **Mapas de Imagen (`<map>` y `<area>`):** Permiten crear zonas clicables dentro de una imagen:

```
<img src="planeta.jpg" alt="Mapa" usemap="#mimapa">
<map name="mimapa">
  <area shape="rect" coords="x1,y1,x2,y2" href="a.html" alt="Zona A">
  <area shape="circle" coords="x,y,radio" href="b.html" alt="Zona B">
</map>
```

(También existe `shape="poly"` con `coords="x1,y1,x2,y2,x3,y3..."`).

GUÍA COMPLETA DE FORMULARIOS HTML5 (TASK 3)

Todos los tipos de <input>, <select>, <textarea>, <button>, atributos de validación y reglas de envío

4. Task 3: Funcionamiento de `<form>` y Reglas de Envío al Servidor — Task 3 • 2 pts

A. Atributos de `<form>`: `action`, `method`, `enctype`

Atributo	Explicación Completa para el Examen
`action="URL"`	Especifica la URL del servidor o script que recibirá y procesará los datos enviados. Si se omite, se envía a la URL de la página actual.
`method="GET"` *(Valor por defecto)*	Los datos se añaden al final de la URL de `action` tras un signo `?`, formando pares `name=value` separados por `&` (ej. `.../avatar2?text=Hi&avatar=123`). **Características:** Datos visibles en la barra de direcciones, se puede guardar en favoritos, tiene límite de longitud (~2048 chars). **No usar para contraseñas ni archivos.**
`method="POST"`	Los datos se envían dentro del **cuerpo (body) de la petición HTTP**. **Características:** Los datos NO aparecen en la URL, no tiene límite de tamaño, permite enviar datos sensibles (contraseñas) y archivos binarios (con `enctype="multipart/form-data"`).

B. CHECKLIST ANTI-TRAMPAS ¿Qué campos SE ENVÍAN y cuáles NO?

Cuando el usuario pulsa Submit, el navegador construye la lista de parámetros `name=value`. Revisa **línea por línea** cada control del formulario con estas 6 reglas:

1. **¿Tiene atributo `name="..."`?** → Si **NO tiene** `name` (aunque tenga `id="..."`), **¡NUNCA SE ENVÍA!** *(Trampa de la línea 20 del examen de ejemplo: `<textarea id="message">`).*
2. **¿Tiene el atributo `disabled`?** → Si tiene `disabled`, el campo está desactivado y **¡NUNCA SE ENVÍA!** *(Trampa de la línea 18 del examen de ejemplo).*
3. **¿Tiene el atributo `readonly`?** → El usuario no puede modificar el texto, pero **SÍ SE ENVÍA** al servidor.
4. **¿Es `type="hidden"`?** → No se ve en la pantalla, pero **SÍ SE ENVÍA SIEMPRE** con el valor de su atributo `value`.
5. **¿Es `checkbox` o `radio`?** → Solo se envía si está **marcado (`checked`)** en el momento de enviar.
6. **¿Es un botón `type="reset"` o `type="button"`?** → **NUNCA se envían**. En cambio, `type="image" name="btn"` **SÍ envía** las coordenadas del clic como `btn.x` y `btn.y`.

5. Catálogo Completo de Controles de Formulario (`<input>`, `<select>`, `<textarea>`, `<button>`) — Referencia Completa

Elemento / Tipo	Cómo se muestra en el navegador	Qué datos envía y detalles importantes
`<input type="text">` *(Tipo por defecto si se omite `type`)*	Campo de texto de **una sola línea**.	Envía el texto escrito (string). Si tiene `value="Inicial"`, aparece pre-rellenado con ese texto.
`<input type="password">`	Campo de texto que **oculta los caracteres** con puntos/asteriscos.	Envía la contraseña en texto plano (por eso requiere `POST` + HTTPS).
`<input type="email">` `<input type="url">` `<input type="tel">`	Campos de texto de una línea (en móviles abren el teclado específico con `@`, `/` o numérico).	• `email`: Valida automáticamente que contenga `usuario@dominio`. • `url`: Valida que empiece por `http://` o `https://`. • `tel`: No impone formato fijo salvo que uses `pattern="..."`.
`<input type="number">` `<input type="range">`	• `number`: Caja numérica con flechas arriba/abajo. • `range`: Barra deslizante (*slider*).	Aceptan los atributos `min="0"` (mínimo), `max="100"` (máximo) y `step="5"` (incremento permitido). Envían un número.
`<input type="date">` / `time` / `color` / `search`	Selector de calendario (fecha), reloj (hora), paleta de color o caja de búsqueda con botón `X`.	• `date` envía formato `YYYY-MM-DD`. • `color` envía código hexadecimal `#rrggbb`.
`<input type="checkbox">`	Casilla de verificación cuadrada (permite marcar **varias opciones independientes**).	Si está marcada (atributo `checked`), envía `name=value` (si no tiene `value`, envía `name=on`). Si está desmarcada, no envía nada.
`<input type="radio">`	Botón circular de opción única. Para que sean mutuamente excluyentes, **todos los radios del grupo DEBEN tener exactamente el mismo atributo `name="..."`**.	Envía el `value="..."` del único botón que esté seleccionado (`checked`) dentro de ese grupo `name`.
`<input type="file">`	Botón "Seleccionar archivo / Browse..." para subir ficheros desde el dispositivo.	Atributo `accept="image/*,.pdf"` filtra tipos de archivo; `multiple` permite elegir varios archivos. Requiere `method="POST"`.
`<input type="hidden">`	**Invisible** para el usuario en la página.	Envía siempre el par `name=value` al servidor (útil para IDs internos, como `name="avatar" value="123"` en el examen de ejemplo).
`<input type="submit">` `<input type="image">` `<input type="reset">` `<input type="button">`	• `submit`: Botón que envía el formulario (texto = `value`). • `image`: Imagen (`src="..."`) que actúa como botón *submit*. • `reset`: Botón que borra/restaura todos los campos a su valor inicial. • `button`: Botón genérico (no hace nada sin JavaScript).	• `submit`: Solo envía su `name=value` si tiene atributo `name` y fue el botón pulsado. • `image`: Envía las coordenadas X e Y donde se hizo clic (`name.x=12&name.y=8`). • `reset`: **No envía el formulario** (solo lo limpia).
`<textarea rows="4" cols="30">Texto inicial</textarea>`	Área de texto de **múltiples líneas**.	**¡CUIDADO!** A diferencia de `<input>`, `<textarea>` tiene etiqueta de cierre y su valor inicial va **entre las etiquetas**, NO en un atributo `value`.
`<select name="pais"> <option value="lv" selected>Latvia</option> </select>`	Lista desplegable (*drop-down list*). Si el `<select>` tiene el atributo `multiple` o `size="4"`, se muestra como una caja con varias opciones visibles.	Envía el atributo `value` del `<option>` elegido (si el `<option>` no tiene atributo `value`, envía el texto contenido dentro de `<option>`). `selected` marca la opción por defecto. `<optgroup label="...">` agrupa opciones con un título.
`<button type="...">`	Botón que permite meter HTML dentro (texto con formato, iconos `<img>`).	**¡TRAMPA!** Si no pones `type` en un `<button>` dentro de un `<form>`, su comportamiento por defecto es `type="submit"` (envía el formulario).

6. Diccionario de Atributos de Formularios

Atributo	Qué hace exactamente
`placeholder="..."`	Muestra un texto de ayuda/ejemplo gris dentro del campo cuando está vacío; desaparece en cuanto el usuario escribe.
`required`	Campo obligatorio: el navegador bloquea el envío del formulario si el usuario lo deja vacío.
`maxlength="100"` `minlength="3"`	Número máximo (o mínimo) de caracteres que el navegador permite escribir en un campo de texto o `textarea`.
`min="1" max="10" step="2"`	Valor numérico/fecha mínimo, máximo e incremento válido (con esos valores solo acepta 1, 3, 5, 7, 9).
`pattern="[A-Za-z]{3}"`	Expresión regular (RegEx) que el valor debe cumplir para poder enviarse.
`disabled` vs `readonly`	• `disabled`: Gris, no recibe foco, **NO se envía al servidor**. • `readonly`: No se puede editar, sí recibe foco, **SÍ se envía al servidor**.
`checked` vs `selected`	• `checked`: Marca por defecto un `checkbox` o `radio`. • `selected`: Marca por defecto un `<option>` dentro de `<select>`.
`autofocus`	Pone el cursor automáticamente en ese campo al cargar la página.

7. `<label>`, `<fieldset>` y `<legend>`

A. Las 2 formas de vincular un `<label>` con un control

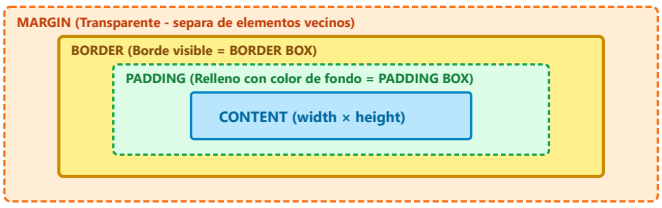
1. **Con el atributo `for` (Explícita — la del examen):**
   `<label for="message">Tu mensaje:</label>`
   `<textarea id="message" name="msg"></textarea>`
   → El valor de `for="..."` **DEBE coincidir exactamente con el** `id="..."` del control (¡no con el `name`!).
2. **Envolviendo el control (Implícita):**
   `<label><input type="checkbox" name="ok"> Acepto</label>`
   → No necesita `for` ni `id` porque el `input` está dentro de `<label>`.

¿Para qué sirve `<label>`? Mejora la accesibilidad y la usabilidad: cuando el usuario **hace clic en el texto del** `<label>`, el navegador enfoca automáticamente el campo de texto o activa el `checkbox` / `radio`.

B. Agrupación con `<fieldset>` y `<legend>`

- `<fieldset>`: Dibuja un recuadro/borde alrededor de un grupo de campos relacionados (si pones `<fieldset disabled>`, deshabilita todos los campos de su interior a la vez).
- `<legend>`: Título incrustado en el borde superior del `<fieldset>`.

8. Task 5: CSS Box Model — Todas las Fórmulas — Task 5 • 2 pts



A. Fórmulas de Tamaño según `box-sizing`

- **1. Por defecto (`box-sizing: content-box`):**
  - Las propiedades CSS `width` y `height` definen **ÚNICAMENTE el Content Box**.
  - **Padding Box Width** = `width + padding-left + padding-right`
  - **Border Box Width (ancho visible)** = `width + padding-left + padding-right + border-left + border-right`
  - **Espacio total ocupado** = `Border Box + margin-left + margin-right`
- **2. Con `box-sizing: border-box`:**
  - Las propiedades `width` y `height` definen el **Border Box completo** (incluyen contenido + padding + border).
  - **Content Width real** = `width - padding-left - padding-right - border-left - border-right`.

B. Reglas de Oro para Cálculos en Task 5

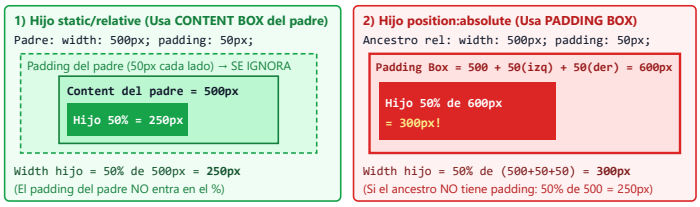
1. **¿Cuándo funcionan `width` y `height`?**
   Funcionan en elementos de bloque (`div, p, h1, footer, article...`), en elementos con `display: block` o `display: inline-block`, en elementos con `position: absolute` o `fixed`, y en elementos reemplazados (`img, input`).
   ¡TRAMPA! Si un `<span>`, `<a>` o `<small>` es `display: inline` (por defecto) y `position: static/relative`, **ignora por completo `width` y `height`** (su tamaño sería solo lo que ocupe su texto). En el examen de ejemplo les ponen `display: block` para que sí apliquen los porcentajes.
2. **¿Qué pasa si un elemento de bloque tiene `width: auto` (no tiene `width` escrito)?**
   Estira su caja automáticamente para ocupar el **100% del Content Width de su padre** (descontando su propio margin, border y padding).
3. **Centrado horizontal de un bloque:** `margin: 0 auto;` (con un `width` menor al 100%) reparte el espacio sobrante a partes iguales entre izquierda y derecha, centrando la caja.
4. **Colapso de márgenes verticales (Margin Collapsing):**
   Si un elemento tiene `margin-bottom: 30px` y el elemento debajo tiene `margin-top: 20px`, la distancia vertical entre ellos **NO es 50px, sino 30px** (el mayor de los dos). Los márgenes horizontales (`left/right`) **NUNCA colapsan** (20px + 30px = 50px).

9. Task 6: Positioning y Containing Block (Todos los Casos) — Task 6 • 2 pts

En las diapositivas **90 a 96 de `04_css_language_2025`**, la profesora explica exactamente cómo cambia el cálculo de porcentajes (`width: 40%`, `top: 10%`) según el valor de `position` y si hay `padding`:

Valor de `position`	¿Sale del flujo normal?	¿Cuál es su Containing Block (sobre qué calcula `%` y `top/left`)?
`static` *(Por defecto)*	**NO**. Ignora `top, bottom, left, right, z-index`.	El **Content Box** (solo `width × height`, **SIN padding**) de su **elemento padre de bloque más cercano**.
`relative`	**NO** (su hueco original se conserva en la página).	• Para `width/height` en `%`: el **Content Box de su padre directo**. • Para `top, left, bottom, right`: se desplaza respecto a **su propia posición original**.
`absolute`	**SÍ** (sale del flujo, no ocupa hueco, se convierte en bloque).	Busca hacia arriba el **ancestro más cercano que tenga `position != static`** (es decir, `relative`, `absolute`, `fixed` o `sticky`). Si no hay ninguno, usa el bloque inicial (`<html>` / viewport). **¡MUY IMPORTANTE!** Calcula sus `%` respecto al **PADDING BOX (`width + padding-left + padding-right`)** de ese ancestro posicionado (Diapositiva 95).
`fixed`	**SÍ** (fijo en pantalla al hacer scroll).	Siempre el **Viewport** (la ventana del navegador: `100vw × 100vh`).
`sticky`	Híbrido	Se comporta como `relative` hasta que al hacer scroll alcanza el umbral (ej. `top: 0`), momento en que se queda fijo como `fixed` dentro de su contenedor.

Ejemplo Comparativo con Padding (Diapositivas 91 y 95)



10. Propiedades de Layout Adicionales: `display` , `float` , `clear` , `z-index` , `overflow` — Diapositiva 04

A. Valores de `display`

- `display: block`: Salto de línea antes y después; acepta `width`, `height`, `margin` y `padding` completos.
- `display: inline`: Fluye en la línea de texto; **ignora `width`, `height` y `margin-top/bottom`**.
- `display: inline-block`: Fluye en la misma línea como un elemento *inline*, pero **SÍ acepta `width`, `height`, `margin` y `padding`** como un bloque.
- `display: none`: Oculta el elemento por completo y **elimina su espacio** en la página (a diferencia de `visibility: hidden`, que lo hace invisible pero **mantiene su hueco vacío**).

B. `float` y `clear`

- `float: left | right | none`: Saca parcialmente al elemento del flujo normal y lo empuja a la izquierda o derecha de su contenedor; el texto y los elementos inline posteriores lo rodean. (También convierte al elemento en bloque, aceptando `width/height`).
- `clear: left | right | both`: Impide que el elemento tenga elementos flotantes a su izquierda, derecha o en ambos lados, forzándolo a **bajar debajo de cualquier elemento con `float`** anterior.

C. `z-index` y `overflow`

- `z-index: número`: Controla qué elemento se dibuja **encima de otro** cuando se solapan (mayor número = más al frente). **¡OJO!** Solo funciona en elementos con `position != static` (o hijos de flex/grid).
- `overflow: visible | hidden | scroll | auto`: Qué ocurre si el contenido de una caja supera su `width/height`:
  - `visible` (por defecto): se sale fuera de la caja.
  - `hidden`: recorta/oculta lo que sobresale.
  - `scroll`: muestra barras de scroll siempre.
  - `auto`: muestra scroll solo si hace falta.

11. Task 7: Tabla Maestra de Selectores Básicos, Combinadores y Atributos — Task 7 • 2 pts

Selector	Tipo	Qué selecciona exactamente
`*`	Universal	Todos los elementos del documento.
`article`	Tipo / Etiqueta	Todos los elementos `<article>`.
`.section`	Clase	Cualquier elemento cuyo atributo `class` contenga la palabra `section` (ej. `class="main section"`).
`article.section` *(Sin espacio)*	Etiqueta + Clase	Elementos `<article>` que **tengan ellos mismos** `class="section"`.
`.a.b` *(Sin espacio)*	Doble clase	Elementos que tengan **ambas clases a la vez**: `class="a b"`.
`#actors`	ID	El elemento único con `id="actors"`.
`A, B`	Agrupación (OR)	Selecciona todos los elementos `A` **Y ADEMÁS** todos los elementos `B`.
`A B` *(Con espacio)*	Descendiente	Cualquier `B` que esté **dentro de** `A` (hijo, nieto, bisnieto...).
`A > B`	Hijo directo (Child)	Cualquier `B` cuyo **padre directo** sea `A` (solo 1 nivel por debajo).
`A + B`	Hermano adyacente	El elemento `B` que va **inmediatamente después** de `A` (compartiendo el mismo padre).
`A ~ B`	Hermano general	Todos los elementos `B` que sean hermanos de `A` y aparezcan **después de** `A` (no importa si hay otros elementos en medio).
`input[value]`	Presencia de atributo	Cualquier `<input>` que tenga escrito el atributo `value` (con cualquier valor).
`input[type="text"]`	Valor exacto	`<input>` cuyo atributo `type` sea exactamente igual a `"text"`.
`[class~="box"]`	Palabra en lista	El atributo contiene la palabra exacta `box` separada por espacios.
`[lang|="en"]`	Prefijo con guion	El valor es exactamente `"en"` o empieza por `"en-"` (ej. `"en-US"`).
`a[href^="https"]`	Empieza por (`^=`)	Enlaces cuyo `href` **empieza por** `"https"` (ej. enlaces seguros o `href^="#"` para enlaces internos).
`a[href$=".pdf"]`	Termina en (`$=`)	Enlaces cuyo `href` **termina en** `".pdf"`.
`a[href*="avatar"]`	Contiene (`*=`)	Enlaces cuyo `href` **contiene la subcadena** `"avatar"` en cualquier parte.

12. Pseudo-clases (:) y Pseudo-elementos (::) — Diapositiva 05

A. Pseudo-clases Estructurales (¡Cuidado con `child` vs `of-type`!)

Pseudo-clase	Cómo cuenta los hijos
`:first-child` `:last-child`	Selecciona un elemento si es el **1er (o último) hijo** de su padre. Ej: `p:first-child` selecciona un `<p>` solo si es el 1er hijo de su contenedor.
`:nth-child(an+b)` `:nth-last-child(...)`	Cuenta **TODOS los hijos del padre** sin importar qué etiqueta tengan (empezando desde **1**; `nth-last-child` cuenta desde el final): • `:nth-child(2n+1)` o `:nth-child(odd)` → **Impares: 1º, 3º, 5º, 7º...** • `:nth-child(2n)` o `:nth-child(even)` → **Pares: 2º, 4º, 6º, 8º...** • `:nth-child(3n)` → **Cada 3: 3º, 6º, 9º...** | `:nth-child(3)` → Solo el **3º**.
`:first-of-type` `:last-of-type` `:nth-of-type(an+b)`	Cuenta **SOLO los hermanos de esa misma etiqueta**. *Diferencia clave en un `<dl>` con pares `dt, dd, dt, dd`:* • `dd:nth-child(2)` selecciona solo el 1er `<dd>` (porque es el 2º hijo del `dl`). • `dd:nth-of-type(2)` selecciona el **2º** `<dd>` (que es el 4º hijo del `dl`).
`:only-child` / `:only-of-type` / `:empty`	• `:only-child`: Hijo único de su padre. • `:empty`: Elemento que no tiene ningún hijo ni texto dentro (ej. `<p></p>`).
`:not(X)`	Selecciona elementos que **NO cumplan** el selector `X` (ej. `input:not([disabled])`).

B. Pseudo-clases de Estado (Enlaces y Formularios)

- **Enlaces (Orden obligatorio LVHA):** `a:link` (no visitado) → `a:visited` (visitado) → `a:hover` (ratón encima) → `a:active` (mientras se hace clic).
- **Formularios:** `:focus` (campo con el cursor activo), `:checked` (checkbox/radio marcado), `:disabled` (con atributo `disabled`), `:enabled` (no deshabilitado), `:required`, `:optional`, `:valid`, `:invalid`.

C. Pseudo-elementos (::)

- `::first-letter`: Primera letra de un bloque de texto (ej. letra capitular grande).
- `::first-line`: Primera línea visual de un párrafo.
- `::before` y `::after`: Insertan contenido generado antes o después del contenido real del elemento usando la propiedad `content: "Texto";`.

D. Cascada, Herencia y Especificidad `(a, b, c)`

Si dos reglas CSS apuntan al mismo elemento y definen la misma propiedad, ¿cuál gana?
1º `!important` gana a todo.
2º **Atributo `style="..."` en el HTML** (vale 1000).
3º **Mayor Especificidad `(a, b, c)`:**
- **a** = Número de selectores de **ID** (`#actors`)
- **b** = Número de **Clases** (`.cast`), **Atributos** (`[value]`) y **Pseudo-clases** (`:hover`, `:nth-child`)
- **c** = Número de **Etiquetas** (`article`, `h1`, `dl`) y **Pseudo-elementos** (`::before`)

(Nota: `:not(X)` no suma por sí mismo, pero sí suma lo que haya dentro de `X`).

4º **Orden en el archivo:** Si empatan en especificidad, **gana la regla escrita más abajo**.

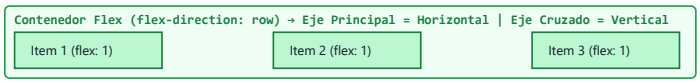
13. Task 4: Diccionario Visual de Propiedades CSS — Task 4 • 2 pts

En la **Task 4** tienes que explicar con palabras cómo cambia el aspecto visual de una sección al aplicar el CSS. Usa esta guía rápida:

Propiedad CSS	Cómo explicarlo en el examen
`font-size: 2em` vs `2rem`	• `2em`: Tamaño de letra = **el doble (2×) del tamaño de fuente de su elemento padre**. (¡Ojo! Si un padre tiene `2em` y su hijo `1.5em`, se multiplican: `2 × 1.5 = 3×`). • `2rem`: El doble del tamaño de fuente del elemento raíz `<html>` (no se acumula).
`width: 100vw` / `height: 100vh`	• `100vw`: 100% del ancho de la ventana (*viewport width*). • `100vh`: 100% de la altura de la ventana (*viewport height*). `vmin` / `vmax` usan el lado menor o mayor de la ventana.
`font-weight: 900 | bold | normal`	Grosor de la fuente (de `100` fina a `900` extra-negrita/black; `normal` = 400, `bold` = 700).
`font-style: italic | normal`	Texto en *cursiva* (`italic` / `oblique`) o normal.
`font-variant: small-caps`	Muestra las letras minúsculas como **VERSALITAS** (mayúsculas más pequeñas).
`text-transform: uppercase | capitalize | lowercase`	• `uppercase`: TODO EN MAYÚSCULAS. • `capitalize`: Primera Letra De Cada Palabra En Mayúscula. • `lowercase`: todo en minúsculas.
`text-decoration: underline | none | line-through`	• `underline`: Texto subrayado. • `none`: Quita el subrayado (muy usado en enlaces `a`). • `line-through`: Texto ~~tachado~~.
`text-align: left | center | right | justify`	Alineación horizontal del texto y elementos inline dentro del bloque (**se hereda**).
`line-height: 1.5`	Altura de línea (interlineado = 1.5 veces el tamaño de la fuente).
`color` vs `background-color`	• `color`: Color del texto (**SÍ se hereda** a los hijos, salvo en enlaces `<a>` que tienen su propio color azul por defecto). • `background-color`: Color de fondo del Content + Padding (**NO se hereda**).
Formatos de Color: `RGBA(0,0,100,0.8)` `#ff0000` / `HSL(...)`	• `RGB(R, G, B)`: cada canal va de `0` a `255` (`(0,0,0)` =negro, `(255,255,255)` =blanco, `(0,0,100)` =azul oscuro). • Canal `A` (Alpha) en `RGBA`: opacidad de `0` (transparente) a `1` (opaco). `0.8` = 80% opaco. • Propiedad `opacity: 0.5`: hace semitransparente **todo el elemento y sus hijos** (mientras que `background: rgba(...)` solo hace transparente el fondo).
`background: url(a.jpg) no-repeat center / cover`	Imagen de fondo sin repetirse, centrada, y escalada para cubrir toda la caja (`cover`) o para caber entera sin recortarse (`contain`).
`list-style-type: none | square | upper-roman`	Cambia o quita (`none`) las viñetas o números de una lista `<ul>` / `<ol>`.
`border-collapse: collapse`	En una `<table>`, fusiona los bordes de celdas contiguas en un único borde simple.

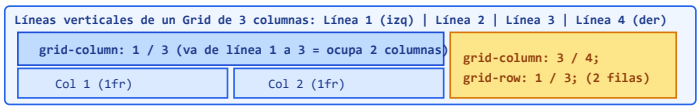
14. Flexbox, CSS Grid y Responsive Design — Diapositivas 06

A. CSS Flexbox (`display: flex` en el contenedor)



- **Propiedades del Contenedor Flex:**
  - `flex-direction: row` (por defecto: fila izq→der) | `row-reverse` | `column` (columna arriba→abajo) | `column-reverse`.
  - `flex-wrap: nowrap` (por defecto: todos en 1 sola línea) | `wrap` (salta a otra línea si no caben).
  - `justify-content` (alinea en el **eje principal**): `flex-start`, `flex-end`, `center`, `space-between` (extremos pegados, espacio igual entre ellos), `space-around`, `space-evenly`.
  - `align-items` (alinea en el **eje secundario/perpendicular**): `stretch` (por defecto: estira), `flex-start`, `flex-end`, `center`, `baseline`.
- **Propiedades de los Hijos (Flex Items):**
  - `order: número` (por defecto `0`; menor número aparece antes visualmente).
  - `flex-grow: número` (cuánto crece para repartirse el espacio libre; ej. si todos tienen `1`, miden igual).
  - `flex-shrink: número` (cuánto se encoge si falta espacio) y `flex-basis: tamaño` (tamaño base inicial). Shorthand: `flex: 1 1 200px`.

B. CSS Grid (`display: grid` en el contenedor)



- `grid-template-columns: 200px 1fr 2fr;` → Crea **3 columnas**: la 1ª fija de `200px`, la 2ª se lleva `1/3` del espacio libre y la 3ª se lleva `2/3`.
- `repeat(3, 1fr)` equivale a `1fr 1fr 1fr` (3 columnas iguales).
- `gap: 10px 20px` (o `grid-gap`): separación de `10px` entre filas y `20px` entre columnas.
- **Posicionar un hijo por líneas:** `grid-column: 1 / 3` (empieza en la línea vertical 1 y acaba en la 3 → **ocupa 2 columnas**). También se puede escribir `grid-column: 1 / span 2`.
- **Por áreas con nombre:** En el padre se define el mapa visual con `grid-template-areas: "head head" "nav main" "foot foot";` y en cada hijo se asigna su zona con `grid-area: head;`.

C. Diseño Responsivo y Media Queries (`06_mobile_2023.pdf`)

- **Etiqueta Viewport:** `<meta name="viewport" content="width=device-width, initial-scale=1.0">` hace que el ancho del viewport coincida con el ancho físico de la pantalla del móvil (sin ella, el móvil simula una pantalla de escritorio de ~980px y se ve todo diminuto).
- **Media Queries (`@media`):** Aplican reglas CSS solo si se cumple la condición de tamaño:
  - `@media screen and (max-width: 600px) { ... }` → Se aplica en pantallas de **600px o menos** (móviles).
  - `@media (min-width: 768px) and (max-width: 1024px) { ... }` → Se aplica entre 768px y 1024px (tablets).

15. Solución Detallada del Examen de Ejemplo (`1test_example.pdf`) — 13 / 13 pts

Tarea	Enunciado y Código Implicado	Respuesta Modelo Completa (10/10)
Task 1: HTML Syntax (1 pt)	Encontrar y corregir el error de sintaxis HTML5 en el primer fragmento (líneas 1–12). *Línea 8:* `<p class="heading"><h1>First</h1></p>`	• **Dónde está el error:** En la **línea 8**. • **Explicación:** El elemento párrafo (`<p>`) solo puede contener contenido *inline* (phrasing content); en HTML5 es ilegal anidar un elemento de bloque como `<h1>` dentro de un `<p>`. • **Corrección:** Eliminar el `<p>` dejando `<h1 class="heading">First</h1>` (o sustituir `<p>` por `<div class="heading"><h1>First</h1></div>`).
Task 2: HTML (2 pts)	1) ¿Qué son los elementos `dl, dt, dd` y cómo se ven? 2) ¿Por qué se usan los símbolos especiales `<<` y `>>` y cómo se ven? 3) Si el documento está en `http://www.avatar.com/movie/html/index.html`, ¿cuál es la URL absoluta de la imagen en la línea 3 (`src="../../images/banner.jpg"`)? 4) ¿Qué ocurre al hacer clic en los enlaces de las líneas 13 (`href="#actors"`) y 28 (`href="mailto:info@avatar.com"`)?	1) **`<dl>`** es una lista de descripción (*description list*), **`<dt>`** define cada término (*description term*, en este caso el personaje) y **`<dd>`** define la descripción/detalle (el actor). Por defecto son elementos de bloque (cada uno en una línea nueva) y `<dd>` aparece con sangría (margen izquierdo) respecto a `<dt>`. 2) Son **entidades de caracteres HTML** que representan los signos menor que (`<`) y mayor que (`>`). Se usan para que el navegador muestre literalmente **`<<Sam>>`** en pantalla en lugar de interpretar `<Sam>` como una etiqueta HTML. 3) El primer `../` sube de `/html/` a `/movie/`; el segundo `../` sube de `/movie/` a la raíz del dominio. La URL absoluta es: **`http://www.avatar.com/images/banner.jpg`** 4) • **Línea 13 (`href="#actors"`):** Es un enlace interno (ancla/fragmento); desplaza (hace scroll) la página actual hasta el elemento con `id="actors"` (el `<article>` de la línea 2). • **Línea 28 (`href="mailto:info@avatar.com"`):** Abre el cliente de correo electrónico predeterminado del usuario creando un nuevo mensaje dirigido a `info@avatar.com`.
Task 3: HTML Form (2 pts)	1) ¿A qué URL y cómo se enviarán los datos del formulario? 2) ¿Qué campos y de qué tipo se enviarán al servidor? 3) ¿Qué hace el elemento `<label>` (línea 19) y cómo se ve? 4) ¿Cómo se ve el elemento `<input>` de la línea 22? 5) ¿Cuál es el significado de los atributos en la línea 20 (`<textarea id="message" placeholder='Message' maxlength="100">`)?	1) Se envían a la URL **`https://www.avatar.com/movies/avatar2`** usando el método HTTP **`GET`** (los pares `name=value` se añaden al final de la URL detrás de `?` separados por `&`). 2) **Campos que SÍ se envían:** • `text` (línea 16): campo de texto de una línea (`type="text"`). • `email` (línea 17): campo de dirección de correo electrónico (`type="email"`). • `avatar` (línea 21): campo oculto (`type="hidden"`) con valor fijo `"123"`. • `avatar2.x` y `avatar2.y` (línea 22): coordenadas X e Y del clic en el botón de imagen (`type="image"`). **Campos que NO se envían:** • Línea 18 (`name="movie"`): **NO se envía** porque tiene el atributo `disabled`. • Línea 20 (`<textarea id="message">`): **NO se envía** porque carece del atributo `name`. 3) Muestra el texto `"Message"` y está asociado al `<textarea>` porque su atributo `for="message"` coincide con el `id="message"` del `textarea`. Al hacer clic sobre el texto del `label`, el cursor (foco) se coloca automáticamente dentro del `textarea`. 4) Se muestra como la imagen `submit.png` y funciona como un botón de envío (*submit button*) del formulario. 5) • `id="message"`: identificador único del elemento en el documento (usado por el `label`). • `placeholder='Message'`: muestra la palabra "*Message*" como pista visual en gris cuando el área de texto está vacía; desaparece al escribir. • `maxlength="100"`: limita la entrada del usuario a un máximo de 100 caracteres.
Task 4: Styles (2 pts)	Describir qué cambiará visualmente en la sección `actors` (líneas 2–12) al aplicar la hoja de estilos CSS dada.	1) **Herencia de `header`:** Como la sección está dentro de `<header>` (`color: white`), todo el texto dentro de ella hereda el **color blanco**. 2) **`<article id="actors" class="section">` (por `.section`):** Tendrá una altura igual al 100% de la altura de la ventana del navegador (`height: 100vh`) y un color de fondo **azul oscuro semitransparente** con 80% de opacidad (`background-color: RGBA(0,0,100,0.8)`). 3) **Imagen del banner (línea 3, por `.banner img`):** La imagen se escalará para ocupar el **100% del ancho** de su contenedor. 4) **Encabezado `<h1>Cast</h1>` (línea 4, por `#actors h1`):** Su tamaño de letra será el doble que el de su padre (`font-size: 2em`), con grosor extra-negrita (`font-weight: 900`), y tendrá un relleno interno (`padding`) de `20px` arriba y abajo y `0px` a izquierda y derecha. 5) **Lista `<dl class="cast">` (línea 5, por `.cast`):** Se convierte en un contenedor **CSS Grid de 2 columnas de igual anchura (`1fr 1fr`)**, colocando cada `<dt>` en la columna izquierda y su correspondiente `<dd>` a su lado en la columna derecha (formando 3 filas de 2 columnas). 6) **Elementos `<dt>` y `<dd>` (líneas 6–11):** Ambos tendrán un tamaño de fuente de `1.5em` y se mostrarán en **versalitas (`font-variant: small-caps`)**. Además, los términos **`<dt>` aparecerán subrayados** (`text-decoration: underline`).
Task 5: Box Model (2 pts)	Calcular el ancho (`width`) y alto (`height`) del enlace `<a>` en la línea 28.	• El contenedor `<footer>` mide **`1000px` de ancho × `500px` de alto**. • `footer small` tiene `display: block` y es hijo de `footer`, por lo que calcula sus porcentajes respecto a `footer`: – Width de `<small>` = 50% de 1000px = **`500px`**. – Height de `<small>` = 80% de 500px = **`400px`**. • `footer a` tiene `display: block` y posición estática por defecto, por lo que calcula sus porcentajes respecto a su padre directo (`<small>`): – **Width de `<a>`** = 40% de 500px = **`200px`**. – **Height de `<a>`** = 60% de 400px = **`240px`**.
Task 6: Positioning (2 pts)	Si se añaden las reglas `footer {position:relative}` y `footer a {position:absolute}`, ¿cuál será el nuevo ancho de `<a>` en la línea 28?	• Al tener `position: absolute`, el bloque contenedor (*containing block*) del enlace `<a>` deja de ser su padre directo `<small>` (porque `<small>` tiene `position: static`) y pasa a ser su **ancestro posicionado más cercano**, que ahora es **`<footer>`** (`position: relative`, de `1000px × 500px`). • Por tanto, el porcentaje de `width: 40%` se calcula directamente sobre los `1000px` del `<footer>`: – **Nuevo Width de `<a>`** = 40% de 1000px = **`400px`** (y su nuevo Height sería 60% de 500px = **`300px`**).
Task 7: Selectors (2 pts)	Especificar qué elementos HTML (y en qué líneas) seleccionan los 4 selectores dados.	1) **`.section`** → Selecciona los 3 elementos que tienen la clase `section`: • **Línea 2:** `<article id="actors" class="section">` • **Línea 14:** `<article class="section">` • **Línea 26:** `<footer class="section">` 2) **`input[value]`** → Selecciona los elementos `<input>` que tienen el atributo `value`: • **Línea 18:** `<input name="movie" value="Avatar 2" disabled>` • **Línea 21:** `<input type="hidden" name="avatar" value="123">` 3) **`article, #article`** → Selecciona todos los elementos `<article>` (o con `id="article"`, que no existe): • **Línea 2:** `<article id="actors" class="section">` • **Línea 14:** `<article class="section">` 4) **`dl :nth-child(2n+1)`** → Selecciona los descendientes de `<dl>` que ocupan una posición de hijo impar (1º, 3º y 5º hijo dentro de `<dl>`), es decir, los tres elementos `<dt>`: • **Línea 6:** `<dt><<Sam>></dt>` (1er hijo) • **Línea 8:** `<dt>Neytiri</dt>` (3er hijo) • **Línea 10:** `<dt>Kiri</dt>` (5º hijo)